Zen: An Instruction-Following Language Model Built on Qwen3-8B

Technical Report v2025.01

Antje Worring, Zach Kelling
Zen LM Research Team
research@zenlm.org

January 2025

Abstract

We present **Zen**, an 8-billion-parameter instruction-following language model that serves as the foundation of the Zen family. Zen is *not* trained from scratch: it is a fine-tuned and repackaged derivative of **Qwen3-8B** [1], the open-weight dense model released by Alibaba's Qwen team under the Apache-2.0 license. Our contribution is post-training and packaging—supervised fine-tuning (SFT) and preference optimization on curated instruction data, together with deployment artifacts (GGUF, MLX, and quantized variants)—rather than a novel architecture or a new pretraining run. We document the inherited Qwen3-8B architecture faithfully, describe the fine-tuning pipeline we actually apply, and direct readers to the upstream Qwen3 technical report for the underlying pretraining methodology and benchmark numbers. This report is intended to give an honest account of what Zen is: a thin, well-documented finetune of a strong open base model.

Contents

1 Introduction **2**

2 Architecture (Inherited from Qwen3-8B) **2**
2.1 Overview . . . 2
2.2 Grouped Query Attention . . . 3
2.3 Rotary Position Embeddings . . . 3
2.4 Feed-Forward Network . . . 3
2.5 Tokenization . . . 3

3 Post-Training Methodology **4**
3.1 Supervised Fine-Tuning . . . 4
3.2 Preference Optimization . . . 4

4 Evaluation **4**

5 Inference and Deployment **5**

6 Related Work **5**

7 Licensing and Attribution **5**

8 Limitations **5**

9 Conclusion **5**

1 Introduction

Foundation language models trained at scale on diverse internet-scale corpora have demonstrated remarkable generalization across tasks without task-specific fine-tuning [3, 4]. Building a competitive base model from scratch, however, requires pretraining compute that is out of reach for most teams. The pragmatic alternative—adopted here—is to start from a strong, permissively licensed open base model and invest effort in post-training, packaging, and deployment.

Provenance. Zen is a derivative of **Qwen3-8B**, an 8.2B-parameter dense decoder-only transformer released by the Qwen team at Alibaba Cloud under the Apache-2.0 license [1, 2]. All architectural choices (layer count, attention configuration, tokenizer, positional encoding) are inherited unchanged from Qwen3-8B. Earlier versions of this report described a bespoke "Zen MoDE" architecture and an independent 3-trillion-token pretraining run; those claims were inaccurate and have been removed. Zen does not introduce a new architecture and was not pretrained from scratch by the Zen LM team.

What Zen adds. Our actual contributions are:

- A supervised fine-tuning (SFT) pass on curated multi-turn instruction–response data, on top of the released Qwen3-8B weights.
- Optional preference optimization (DPO-style) on a smaller preference set to sharpen instruction adherence and refusal behavior.
- Packaged, ready-to-deploy artifacts: BF16 SafeTensors, GGUF (llama.cpp) quantizations, and MLX builds for Apple Silicon, with documented deployment recipes.

Zen occupies the entry tier of the Zen family. Other Zen language models are likewise finetunes of open Qwen3 base models at different parameter scales (see the Zen family overview).

2 Architecture (Inherited from Qwen3-8B)

2.1 Overview

Zen inherits the Qwen3-8B architecture without modification: a dense decoder-only transformer with grouped-query attention, SwiGLU feed-forward blocks, RMSNorm, and rotary position embeddings. Table 1 reproduces the Qwen3-8B hyperparameters as published in the upstream model configuration [2].

Table 1: Architecture hyperparameters, inherited unchanged from Qwen3-8B [2].

Hyperparameter	**Value**
Parameters (total)	8.2B
Non-embedding parameters	6.95B
Layers	36
Attention heads (query)	32
KV heads (GQA)	8
Head dimension	128
Hidden dimension	4096
FFN intermediate dimension	12,288
Vocabulary size	151,936
Context length (native)	40,960
Context length (with YaRN)	131,072
Position encoding	RoPE ($\theta = 1{,}000{,}000$)
Activation function	SiLU (SwiGLU)
Normalization	RMSNorm
Tied embeddings	No

2.2 Grouped Query Attention

Qwen3-8B employs Grouped Query Attention (GQA) [5] with 32 query heads and 8 key-value heads (head dimension 128). This reduces the KV cache memory footprint by 4× relative to multi-head attention at equivalent query capacity. Attention is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \tag{1}$$

where each group of query heads shares a single key and value projection.

2.3 Rotary Position Embeddings

Rotary position embeddings (RoPE) [6] encode position as a rotation applied to query and key vectors. Qwen3-8B uses a RoPE base frequency of $\theta = 1{,}000{,}000$ and a native maximum position of 40,960 tokens, extensible to 131,072 tokens via YaRN scaling [8] as documented upstream [1].

2.4 Feed-Forward Network

Each transformer layer contains a SwiGLU [7] feed-forward block:

$$\text{FFN}(x) = \left(\text{SiLU}(xW_{\text{gate}}) \odot xW_{\text{up}}\right) W_{\text{down}} \tag{2}$$

with an intermediate dimension of 12,288.

2.5 Tokenization

Zen uses the Qwen3 byte-pair encoding (BPE) tokenizer unchanged, with a vocabulary of 151,936 tokens and the standard Qwen chat template (system, user, and assistant turn markers). We do not retrain or extend the tokenizer.

3 Post-Training Methodology

This section describes what the Zen LM team actually does on top of the released Qwen3-8B weights. We do *not* perform large-scale pretraining; for the pretraining corpus, scale, and procedure, see the upstream Qwen3 technical report [1].

3.1 Supervised Fine-Tuning

Starting from Qwen3-8B, we apply supervised fine-tuning on a curated mixture of multi-turn instruction–response data spanning:

- General question answering and open-ended generation
- Multi-turn dialogue with persona consistency
- Code generation and debugging
- Summarization and document analysis
- Step-by-step reasoning traces

SFT trains with a low learning rate (on the order of 2×10^{-5}) for a small number of epochs, packing sequences and computing loss only on assistant-turn tokens (response masking). The goal is to adapt formatting and instruction-following behavior, not to alter the base model's knowledge.

3.2 Preference Optimization

We optionally apply Direct Preference Optimization (DPO) [13] on a smaller set of preference pairs to improve instruction adherence and refusal calibration. DPO avoids the infrastructure overhead of online RLHF [11, 12] while still optimizing a preference objective:

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}_{(x,y_w,y_l)}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right] \tag{3}$$

with the reference policy π_{ref} set to the SFT checkpoint.

4 Evaluation

On benchmark numbers. We do not report fabricated head-to-head benchmark tables. The capabilities of Zen are, to first order, those of its base model Qwen3-8B; for rigorous, independently reproducible benchmark results on the Qwen3 base and instruct models—covering MMLU, GSM8K, MATH, HumanEval, multilingual evaluation, and long-context retrieval—we refer the reader to the official Qwen3 technical report [1] and the Qwen3-8B model card [2]. Any task-specific numbers we publish for a given Zen release are measured on the released artifact under a stated harness and are reported alongside the corresponding Qwen3-8B baseline so that the effect of our fine-tuning is transparent. We deliberately avoid quoting numbers we have not measured.

Inheritance, not improvement, by default. Light SFT and DPO on a strong base model typically change instruction-following style and formatting more than raw knowledge or reasoning accuracy. We therefore do not claim that Zen exceeds Qwen3-8B on standard knowledge benchmarks absent a measured, attributed comparison.

5 Inference and Deployment

Because Zen is architecturally identical to Qwen3-8B, it runs on any inference stack that supports Qwen3 (Hugging Face Transformers, vLLM, llama.cpp via GGUF, and MLX). At 8B parameters the model fits comfortably on a single 24 GB consumer GPU (e.g., RTX 4090) in BF16, and runs on commodity hardware in 4-bit quantization. We publish the following artifacts:

- BF16 SafeTensors (reference weights).
- GGUF quantizations (Q4_K_M, Q8_0) for llama.cpp on CPU/GPU.
- MLX 4-bit builds for Apple Silicon.

We do not report fabricated throughput tables here; measured throughput depends on the chosen runtime, quantization, and hardware, and matches that of any equivalent Qwen3-8B deployment.

6 Related Work

Zen sits within the now-standard practice of taking a permissively licensed open base model and adapting it via instruction tuning. Dense decoder-only transformers have been the dominant paradigm since GPT-3 [3], and the LLaMA series [9] popularized open base models at the 7–8B scale. The Qwen3 series [1] that underlies Zen continues this lineage with GQA, SwiGLU, RoPE, and a large multilingual tokenizer. Instruction tuning via SFT [10] and preference optimization via RLHF [11] and DPO [13] are the standard post-training steps that Zen applies.

7 Licensing and Attribution

Qwen3-8B is released by Alibaba Cloud under the Apache-2.0 license, which permits commercial use, modification, and redistribution subject to attribution and the terms of the license. Zen, as a derivative, is distributed under the same Apache-2.0 terms with the required upstream attribution and NOTICE. Users should consult the upstream Qwen3-8B model card and LICENSE for authoritative terms.

8 Limitations

Zen inherits the limitations of Qwen3-8B and of autoregressive language models generally. The model can hallucinate plausible-sounding but incorrect facts, particularly for low-frequency knowledge; reasoning degrades on long multi-step problems; and alignment is incomplete, so adversarial prompts can elicit policy-violating outputs. Our light post-training does not remove these limitations, and in some cases fine-tuning can introduce regressions relative to the base model. Because Zen's knowledge and core capabilities derive entirely from Qwen3-8B, any limitation of the upstream model is also a limitation of Zen.

9 Conclusion

Zen is an honest, lightly post-trained, well-packaged derivative of the Apache-2.0 Qwen3-8B model. It does not introduce a new architecture and was not pretrained from scratch; its value lies in curated instruction tuning and convenient deployment artifacts. We document the inherited Qwen3-8B architecture transparently and defer to the upstream Qwen3 technical report for pretraining details and rigorous benchmark numbers. Future Zen releases will continue to build on open Qwen3 base models, with all provenance and licensing stated explicitly.

References

[1] Qwen Team, Alibaba Cloud, "Qwen3 Technical Report," *arXiv:2505.09388*, 2025.

[2] Qwen Team, "Qwen3-8B model card and configuration," Hugging Face, https://huggingface.co/Qwen/Qwen3-8B, 2025.

[3] T. Brown et al., "Language Models are Few-Shot Learners," *NeurIPS*, 2020.

[4] J. Wei et al., "Emergent Abilities of Large Language Models," *TMLR*, 2022.

[5] J. Ainslie et al., "GQA: Training Generalized Multi-Query Transformer Models," *EMNLP*, 2023.

[6] J. Su et al., "RoFormer: Enhanced Transformer with Rotary Position Embedding," *arXiv:2104.09864*, 2021.

[7] N. Shazeer, "GLU Variants Improve Transformer," *arXiv:2002.05202*, 2020.

[8] B. Peng et al., "YaRN: Efficient Context Window Extension of Large Language Models," *arXiv:2309.00071*, 2023.

[9] H. Touvron et al., "LLaMA: Open and Efficient Foundation Language Models," *arXiv:2302.13971*, 2023.

[10] J. Wei et al., "Finetuned Language Models Are Zero-Shot Learners," *ICLR*, 2022.

[11] L. Ouyang et al., "Training Language Models to Follow Instructions with Human Feedback," *NeurIPS*, 2022.

[12] J. Schulman et al., "Proximal Policy Optimization Algorithms," *arXiv:1707.06347*, 2017.

[13] R. Rafailov et al., "Direct Preference Optimization: Your Language Model is Secretly a Reward Model," *NeurIPS*, 2023.